



UNIVERSITÉ DE
SHERBROOKE

MODULE- Sécurité informatique

Guide de l'étudiant

APP1 - S8

GEI-771 - Programmation sécurisée

Semaines 1 et 2

Département de génie électrique et de génie informatique
Faculté de Génie

Automne 2026

Note : En vue d'alléger le texte, le masculin est utilisé pour désigner les femmes et les hommes.

Document gei771.pdf
Version 2.0, Septembre 2026, par Guy Lépine

Réalisé à l'aide de $\text{\LaTeX}$ et TeXstudio

Copyright © 2026 Département de génie électrique et de génie informatique, Université de Sherbrooke.

Table des matières

1	Éléments de compétences visés par l'unité	5
2	Énoncé de la problématique	5
	(1)	5
	(2)	5
	(3)	5
	(4)	6
	(5)	6
	(6)	6
	(7)	6
3	Connaissances à acquérir par la résolution de cette problématique	7
4	Références	8
5	Sommaire des activités liées à la problématique	9
6	Productions à remettre (équipes de deux)	9
7	Semaine 1 - Formation à la pratique procédurale	10
7.1	Question 1	10
7.2	Question 2	10
7.3	Question 3	10
7.4	Question 4	10
7.5	Question 5	11
7.6	Question 6	11
7.7	Question 7	11
7.8	Question 8	11
7.9	Question 9	11
7.10	Question 10	11
7.11	Question 11	13
7.12	Question 12	14
7.13	Question 13	14
7.14	Question 14	14
7.15	Question 15	14
8	Semaine 1 - Formation à la pratique en laboratoire	14
8.0	Activité 0: Actions pré-requises	15
8.0.1	Installation de l'outil de rétro-ingénierie	15
8.0.2	Installation des outils d'obfuscation	15
8.0.3	Familiarisation avec la compilation dotnet	16
8.1	Activité 1 : Rétro-ingénierie d'un programme en clair	16
8.2	Activité 2 : Obfuscation partielle	16

8.3	Activité 3 : Rétro-ingénierie d'un programme obfusqué	17
8.4	Activité 4 : Obfuscation complète et résultats comparatifs . . .	17
8.5	Activité 5: Utilisation de la compilation AOT	18
8.6	Activité 6 : Utilisation des options de compilation	18
8.6.1	Fonctions non-sécuritaires	18
8.6.2	Scénario de base	19
8.6.3	<i>Basic Runtime Checks</i>	19
8.6.4	<i>Buffer Security Check</i>	19
8.6.5	Options SDL	20
8.7	Activité 7: Calcul de la couverture de code par les tests	20
8.8	Activité 8: Création de la nomenclature logicielle	20
8.9	Activité 9: Ajout de mesures d'atténuation à un modèle de menace	21
9	Semaine 2 - Formation à la pratique procédurale	22
9.1	Question 1	22
9.2	Question 2	23
9.3	Question 3	23
9.4	Question 4	23
9.5	Question 5	23
9.6	Question 6	23
10	Semaine 2 - Validation pratique de la solution en laboratoire	26
11	Semaine 2 - Travail de rédaction des productions exigées	26
12	Semaine 2 - 2e rencontre de tutorat: validation des connaissances acquises	27
13	Rapport d'APP	27
13.1	Consignes pour la préparation du dépôt du rapport et de la solution à la problématique	27
13.2	Modalités d'évaluation du rapport d'APP	27
14	Évaluation sommative	28
15	Évaluation de l'unité	28
15.1	Utilisation de l'intelligence artificielle générative	28

1 Éléments de compétences visés par l'unité

GEI-771 - Programmation sécurisée

1. Intégrer la sécurité dans les étapes du processus de développement logiciel
2. Identifier les risques et analyser la conformité d'une application aux standards de sécurité

2 Énoncé de la problématique

(1) Lors de vos activités de consultation en développement logiciel, la compagnie Coup de Sonde décide de faire affaire avec vous afin d'appliquer vos connaissances au développement d'un logiciel de sondage. Il est primordial de concevoir un logiciel intégrant les concepts de programmation sécurisée, qui permettra de garantir la validité des résultats de votes. La compagnie vous fournit les sondages en format texte.

(2) Le développement web a évolué dans plusieurs directions et une avenue populaire est le développement d'interface de programmation d'application (API). Pour les entreprises, ceci amène des sources de revenus supplémentaires en vendant des accès à ces API, souvent même utilisés pour leur propre développement d'applications. La compagnie énonce donc le requis ferme que le logiciel de sondage possède un API évolutif auquel on pourra intégrer des applications externes et vendre des clefs d'API.

(3) La compagnie a un historique de développement avec les technologies Microsoft, qui comprend entre autres les composants pour le développement de services web (SOAP) ou d'API (REST). Les API REST ont pris le pas sur les services web SOAP. Aussi, l'utilisation des récentes versions de dotnet permet de développer des applications compatibles avec les systèmes d'exploitation communs. La compagnie voulant rester à l'avant-garde, l'application de sondage devra donc comprendre un API REST en dotnet, véhiculant l'information dans le format JSON. À des fins de démonstration, utilisez l'outil Postman pour interagir avec votre API. Vous devrez composer une collection de requêtes servant à démontrer que les différents cas d'utilisation sont couverts. Finalement, afin de s'assurer d'un bon niveau d'utilisabilité de l'application, vous devrez fournir une interface Swagger, qui expose aussi un point d'entrée Open API.

(4) Le langage C# est le langage phare du *framework* dotnet de Microsoft. C'est un langage géré, donc plusieurs pièges de programmation, tels que les dépassements de tampons et d'entiers sont gérés de façon interne. Cependant, d'autres pièges doivent être gérés par le programmeur. Pour ces raisons, la compagnie vous demande donc que tout le développement se fasse en C#.

(5) Il est très important de s'assurer que le code qui sera produit soit testé de façon convenable. La livraison de votre code devra contenir des tests unitaires développés à l'aide du *framework* xUnit et montrer une couverture de branche de 100%. Pour vous aider à démontrer la couverture des tests, vous utiliserez l'intégration coverlet incluse automatiquement dans la création de projet xUnit et vous utiliserez l'outil ReportGenerator pour la génération de rapport.

(6) Pour les langages non-gérés, Microsoft a intégré des mécanismes de protection, tels que l'empêchement de l'exécution de la pile. Leur système d'exploitation Windows intègre aussi des mécanismes de sécurité variés selon les versions, mais celui qui est le plus intéressant pour les programmeurs est sans doute la mise en mémoire (sur tas) aléatoire, ou *Address Space Layout Randomization* (ASLR). D'autres systèmes d'exploitation intègrent une forme ou une autre d'ASLR. Étant donné que les applications en dotnet peuvent être exécutées sur différents systèmes d'exploitation, vous devrez en recommander un au point de vue des mécanismes de sécurité qui y sont intégrés.

(7) Vous sentez que votre client a fait quelques recherches en amont lorsqu'il vous fait mention de ses requis au niveau de la sécurité. Le projet devra adopter le processus du *Security Development Lifecycle*. Les pratiques associées à chaque étape devront aussi être étudiées, telles que les analyses de la menace, en utilisant la méthode STRIDE, et les vecteurs d'attaque à la conception et leur identification à l'aide de diagrammes de flux de données (DFD), l'analyse statique à la réalisation et les pratiques de vérification comme les tests dynamiques et de fuzzing. Cependant, avant toute chose, il est pertinent de bien cerner les besoins en sécurité que l'application nécessitera. Ils sont aussi conscients qu'ils pourraient se faire voler leur propriété intellectuelle et veulent que vous recommandiez la configuration à utiliser avec Dotfuscator. Enfin, les dirigeants veulent aussi s'assurer de la sécurité de la chaîne d'approvisionnement logicielle et vous demandent un rapport Cyclone DX pour documenter la nomenclature logicielle (ou *software bill of materials*).

Il vous faudra donc fournir les livrables suivants:

1. un API sécurisé par https et dont l'autorisation se fait par une clef d'accès;

2. une documentation d'API suivant le standard OpenAPI dont l'intégration est faite en utilisant Swagger;
3. une collection de requêtes Postman démontrant les différentes fonctionnalités de votre API, liée à votre schéma d'API OpenAPI;
4. une authentification des participants pour garantir l'unicité de leur participation;
5. une batterie de tests automatisés xUnit en dotnet inclus dans la même solution que le projet d'API qui prouvera une couverture complète du code et l'intégration des mesures de mitigation proposées lors de l'analyse de la surface d'attaque;
6. une analyse d'impact de la sécurité, comprenant une analyse des vecteurs d'attaque;
7. inclure des mécanismes de sécurité lors de la programmation, tels que l'empêchement de l'exécution de la pile, les options d'ajouts de robustesse fournis par Visual Studio;
8. les techniques de protection du code par l'obfuscation et la configuration à utiliser;
9. les recommandations au sujet de l'opération de l'API (architecture système);
10. un rapport de nomenclature logicielle Cyclone DX; et
11. un processus d'identification et de publication des bogues de sécurité.

Notez que pour alléger le développement, une simple persistance des données de sondage dans un système de fichiers (questions et réponses) est requise.

3 Connaissances à acquérir par la résolution de cette problématique

Connaissances déclaratives: QUOI

- Analyse et modélisation des risques d'une application
- Identification des types de failles

Connaissances procédurales: COMMENT

- Mesures de contingence: appels à bannir, protection de la pile, protection des communications, protection des données, etc.
- Niveau de protection des langages

Connaissances conditionnelles: QUAND

- Méthodes de test (analyse statique, dynamique, fuzzing)
- Pièges de la programmation

4 Références**Bibliographie**

- [Cor23] Microsoft Corporation. Microsoft security development lifecycle. *MSDN*, 2023. Voir: <https://www.microsoft.com/en-us/securityengineering/sdl>.
- [How06] M. Howard. *The Security Development Lifecycle*. Microsoft Press, 2006.
- [How07] M. Howard. *Writing Secure Code for Windows Vista*. Microsoft Press, 2007.
- [How10] M. Howard. *24 Deadly Sins of Software Security*. McGraw Hill, 2010.
- [How11] M. Howard. Threat modeling. 2011.
- [Mic11] Microsoft. Basics of secure design, development and test. Technical report, Microsoft, 2011. Voir: <http://download.microsoft.com/download/9/3/5/935520EC-D9E2-413E-BEA7-0B865A79B18C/Basics%20of%20Secure%20Design%20Development%20Test.ppsx>.
- [Net] Palo Alto Networks. What is zero trust architecture. Voir: <https://www.paloaltonetworks.com/cyberpedia/what-is-a-zero-trust-architecture>.
- [Tay08] J. Taylor. Jason taylor on security testing. Technical report, Microsoft, 2008. Voir: <https://www.microsoft.com/en-us/research/blog/a-brief-introduction-to-fuzzing-and-why-its-an-important-tool-for-dev>

5 Sommaire des activités liées à la problématique

Activités de la semaine 1:

- 1^{ère} rencontre de tutorat;
- Étude personnelle et exercices: étude des sujets issus des objectifs d'étude du tutorat;
- Formation à la pratique procédurale: *SDL, modélisation de menaces, protections offertes par le compilateur et l'éditeur de lien*
- Formation à la pratique en laboratoire: *Obfuscation, Options de protection à la compilation*

Activités de la semaine 2:

- Étude personnelle et exercices: sujets issus des objectifs d'étude du tutorat;
- Formation à la pratique procédurale: *Fuzzing, tests de pénétration, péchés de programmation, développement sous compte d'administrateur, niveaux d'intégrités*
- Validation pratique des éléments de solution;
- Rédaction du rapport d'APP;
- 2^{ième} rencontre de tutorat: validation des connaissances acquises;
- Remise du rapport d'APP et des schémas;
- Évaluation formative;
- Évaluation sommative.

6 Productions à remettre (équipes de deux)

Rapport d'APP à remettre avant le début de la deuxième rencontre de tutorat, selon les consignes de rédaction du rapport de l'unité d'APP données à la fin de ce document (section 13).

Inscrire votre nom et votre numéro de matricule sur chaque document remis.

7 Semaine 1 - Formation à la pratique procédurale

But de l'activité

Le but de cette activité est de se familiariser avec le processus de développement SDL (*Security Development Lifecycle*) et plus particulièrement avec la modélisation de la menace.

Lectures préalables:

1. [Cor23]
2. [How06, chapitre 11]
3. [How06, extrait du chapitre 9]
4. [How07, chapitre 3]

7.1 Question 1

Quel est le but de la méthodologie SDL, quelles sont les étapes de celle-ci selon les pratiques qui sont décrites et comment s'intègrent-elles dans les étapes d'un développement logiciel classique?

7.2 Question 2

Quelle est la pratique associée à la phase de formation, et la valeur qu'elle ajoute au SDL? Donnez des exemples de mise en application en entreprise.

7.3 Question 3

Quelles sont les pratiques associées à la phase d'évaluation des besoins, et quelles valeurs ajoutent-elles au SDL? Donnez des exemples de requis de sécurité qui ne sont pas associés à des fonctionnalités d'un logiciel et qui feraient donc partie d'une liste de *requis non-fonctionnels*.

7.4 Question 4

Quelles sont les pratiques associées à la phase de conception, et la valeur qu'elles ajoutent au SDL?

7.5 Question 5

Quelles sont les pratiques associées à la phase de réalisation, et la valeur qu'elles ajoutent au SDL?

7.6 Question 6

Quelles sont les pratiques associées à la phase de contrôle de la qualité, et la valeur qu'elles ajoutent au SDL?

7.7 Question 7

Quelles sont les pratiques associées à la phase de livraison, et la valeur qu'elles ajoutent au SDL?

7.8 Question 8

Quel est la pratique associée à la phase de réponse, et la valeur qu'elles ajoutent au SDL?

7.9 Question 9

En quoi consiste la modélisation de la menace?

7.10 Question 10

Mise en contexte 6 [How11]:

iNTEGRITY est un outil permettant de vérifier l'intégrité des fichiers sur un système (voir la figure 1). Il permet aussi de déterminer si certains fichiers ou certaines clés de registre ont été modifiés depuis la dernière exécution. Ces fonctionnalités utilisent les mécanismes suivants :

- Le nom des fichiers ou des clés de registre
- La grosseur des fichiers ou des clés de registre
- La dernière modification effectuée sur un fichier
- La somme de contrôle (hachage)

Conceptuellement, le programme est divisé en deux parties : une partie opérant sur un ordinateur hôte et une autre faisant office de console d'administration. Ainsi, un utilisateur peut communiquer avec plusieurs ordinateurs distants au lieu de devoir exécuter iNTegrity localement sur chacun des ordinateurs qu'il veut analyser. Lorsqu'elles fonctionnent sur deux machines distinctes, les deux parties communiquent entre elles par TCP/IP.

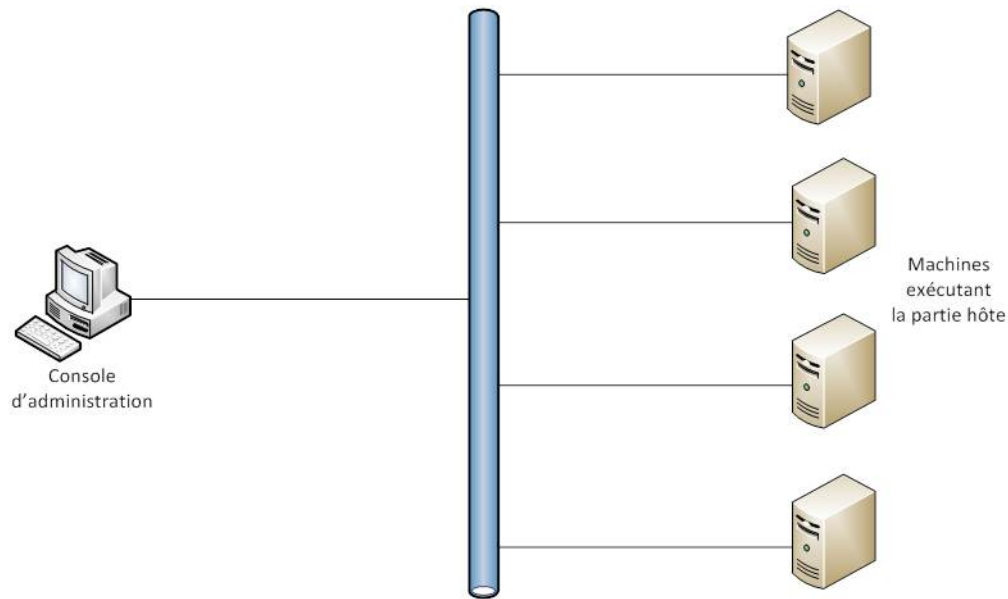


Figure 1: L'outil iNTegrity

La partie hôte

La partie hôte est écrite en C++ et s'exécute en tant que service sur une machine Windows. Son unique rôle est de recevoir des requêtes de la part de la console d'administration et de répondre à ces requêtes. La partie hôte ne fait aucune analyse elle-même, elle envoie plutôt les données brutes directement à la console d'administration qui effectuera l'analyse.

La console d'administration

La console d'administration est écrite en C#. Elle reçoit et analyse les informations obtenues d'une ou de plusieurs parties hôte. Elle possède une base de données enregistrant toutes les informations reçues de la part d'un hôte à chaque fois qu'une requête est effectuée. Un utilisateur peut donc demander à la console de se connecter à un hôte, recevoir les informations brutes quant à l'intégrité de cet hôte, comparer ces données avec celles précédemment enregistrées par rapport à cet hôte et déterminer si quelque chose a changé. Un fichier XML permet de choisir et sauvegarder le type d'informations que l'on désire recevoir de la part de la partie hôte, mais il n'est pas possible de le modifier directement au travers de la console.

Questions :

1. Identifier le(s) entité(s) externe(s) (ou acteurs) présente(s).
2. Identifier le(s) processus à l'oeuvre.
3. Identifier le(s) entrepôt(s) de données dans ce problème.
4. Identifier les flux d'informations entre les différentes entités. Pour vous aider, débutez par penser avec quel(les) entité(s) le(s) entité(s) externe(s) peuvent communiquer. Par la suite, répétez l'exercice avec le(s) processus.
5. Concevoir le diagramme de flux de données avec les informations des exercices précédents. N'oubliez pas d'indiquer les frontières de confiance entre les différentes entités.
6. Identifier les types de menaces potentielles pour chacune des entités. Servez-vous du tableau suivant pour vous aider à répondre à cette question (référé en anglais par l'acronyme STRIDE).

Type de Menace	Mitigation	Ent. Ex-terne	Processus	Flux	Entrepôt de données
(S) Usurpation d'identité	Authentification	X	X		
(T) Falsification	Intégrité		X	X	X
(R) Répudiation	Non répudiation	X	X		X*
(I) Divulgarion d'informations	Confidentialité		X	X	X
(D) Refus de service	Disponibilité		X	X	X
(E) Élévation de privilèges	Autorisation		X		

*Ne s'applique pas toujours

7. Choisir trois entités de type différent (un entrepôt de données, un processus et un flux de données) et donner un exemple concret d'une mesure de mitigation qui pourrait être mise en place afin de contrer une menace précise.

7.11 Question 11

Décrivez le mécanisme et les avantages de la protection de la pile à l'aide de vérification de dépassement de mémoire tampon (*buffer overflow*) à la phase de réalisation d'un logiciel. Indiquez avec quelles options du compilateur ou de l'éditeur de lien il est possible d'activer ces fonctionnalités.

7.12 Question 12

Décrivez le mécanisme et les avantages de la gestion sécuritaire des exceptions à la phase de réalisation d'un logiciel. Indiquez avec quelles options du compilateur ou de l'éditeur de lien il est possible d'activer ces fonctionnalités.

7.13 Question 13

Décrivez le mécanisme et les avantages de l'interdiction d'exécution de données (ou *Data Execution Prevention*) à la phase de réalisation d'un logiciel. Indiquez avec quelles options du compilateur ou de l'éditeur de lien il est possible d'activer ces fonctionnalités.

7.14 Question 14

Décrivez le mécanisme et les avantages d'activer l'option `/sdl`

7.15 Question 15

Décrivez le mécanisme et les avantages de la distribution aléatoire de l'espace d'adressage (ou *Address Space Layout Randomization*) à la phase de réalisation d'un logiciel. Indiquez avec quelles options du compilateur ou de l'éditeur de lien il est possible d'activer ces fonctionnalités.

8 Semaine 1 - Formation à la pratique en laboratoire

Dans cette activité, on collabore par équipe de deux.

But de l'activité

Le but de ce laboratoire est de mettre en pratique des techniques de programmation sécurisée.

Description du laboratoire:

Le laboratoire se concentre sur les principes de rétro-ingénierie d'un programme dotnet et les contre-mesures d'obfuscation qui permettent de les empêcher.

Dans ce laboratoire, vous devrez:

- effectuer la rétro-ingénierie d'un programme dotnet C# en clair

- obfusquer partiellement le programme
- identifier les changements dans la rétro-ingénierie
- obfusquer complètement le programme et remarquer l'impact sur la rétro-ingénierie
- comparer l'impact des différentes méthodes sur la rétro-ingénierie

Vous découvrirez que des *assemblies* faites avec dotnet sont intrinsèquement faciles à décompiler.

8.0 Activité 0: Actions pré-requises

Le laboratoire est une partie de ping-pong entre les stratégies d'obfuscation et de rétro-ingénierie. Il faut donc commencer par l'installation des outils.

Notez que le laboratoire a été monté avec l'utilisation de Visual Studio 2026 et que pour les utilisateurs de PC non-Windows, l'utilisation de VS Code peut s'avérer possible, mais certains outils de compilation et décompilation ne sont offerts que sur Windows.

8.0.1 Installation de l'outil de rétro-ingénierie

Plusieurs outils de rétro-ingénierie existent, mais nous allons utiliser dotPeek, qui est gratuit et efficace. Allez sur le site de JetBrains pour installer la version appropriée pour le système que vous utilisez.

8.0.2 Installation des outils d'obfuscation

Plusieurs outils d'obfuscation existent, mais leurs fonctionnalités les plus intéressantes sont souvent payantes.

Nous allons utiliser les outils suivants:

- Dotfuscator. Il existe une version gratuite (Community), dont les possibilités sont limitées et un essai pour la version professionnelle est disponible
- Obfuscator. L'outil est gratuit et *ouvert*. L'intégration se fait par un *package nuget*

8.0.3 Familiarisation avec la compilation dotnet

Afin de vous mettre à niveau si vous n'êtes pas avec le framework dotnet, prenez le temps de lire les deux références suivantes et d'utiliser les outils *ilasm.exe* et *ildasm.exe* :

- processus d'exécution managée
- Assemblys dans dotnet
- Métadonnées et composants autodescriptifs

8.1 Activité 1 : Rétro-ingénierie d'un programme en clair

La première activité de ce laboratoire est de prendre un programme C# compilé et de le désassembler, puis d'en récupérer le code source. Afin de bien juger du résultat de la décompilation, nous compilerons le code source.

1. récupérez l'archive d'obfuscation du laboratoire
2. Compilez le projet avec la configuration *Debug* et localisez le répertoire de résultat de compilation
3. Démarrez dotPeek et ajoutez le répertoire
4. Faites une comparaison visuelle du code source original et du résultat de la décompilation

Que peut-on conclure de la sécurité d'un programme dotnet *en clair*, i.e. sans obfuscation? Et le résultat de la compilation avec la configuration Release?

8.2 Activité 2 : Obfuscation partielle

Nous allons maintenant utiliser un outil d'obfuscation pour ajouter un niveau de difficulté à la décompilation. Refaites les étapes de l'activité précédente, mais incorporez l'étape suivante à la fin de la compilation du programme:

1. utilisez un outil d'obfuscation
2. compilez le programme
3. naviguez pour donner l'*assembly* Obfuscation.dll (selon le framework) en intrant (la version qui correspond à la configuration d'obfuscar quand vous l'avez configuré)

4. ajouter la protection
5. refaites le même processus, mais en l'intégrant dans les étapes de compilation de la configuration *Release*
6. refaites les mêmes étapes pour les autres outils

Que peut-on conclure de l'exécution du programme protégé?

Y a-t-il un outil plus approprié qu'un autre?

8.3 Activité 3 : Rétro-ingénierie d'un programme obfusqué

Afin de bien saisir le résultat de l'étape supplémentaire faite dans l'activité précédente, refaites les étapes de décompilation et comparer les résultats avec le code source original pour tous les outils d'obfuscation que vous avez utilisés. Comparez aussi le résultat de la décompilation du programme non-obfusqué.

Remarquez attentivement l'impact de l'utilisation de l'option de la suppression des attributs ILDASM des deux outils. C'est surtout cette option qui rendra le code beaucoup moins lisible avec dotPeek.

8.4 Activité 4 : Obfuscation complète et résultats comparatifs

Nous allons maintenant passer à l'étape ultime de l'obfuscation d'un programme .Net dans le cadre du laboratoire. Prenez note que nous faisons l'utilisation d'outils gratuits et que les alternatives commerciales procurent parfois des résultats supérieurs qui sont à considérer lors du choix du programme d'obfuscation.

Plusieurs techniques peuvent être utilisées lors de l'obfuscation d'un programme :

- renommer les variables
- changer le flot d'exécution
- encrypter le code source

Refaites les étapes d'obfuscation et de rétro-ingénierie, en utilisant la version Pro de Dotfuscator. Vous verrez aussi que certaines protections ne sont pas disponibles pour des compilations ciblant .Net.

8.5 Activité 5: Utilisation de la compilation AOT

La compilation AOT (Ahead-of-Time) est une technique qui permet de compiler le code source en code machine avant l'exécution, plutôt que de le compiler à la volée (JIT). Cela peut améliorer les performances et réduire la surface d'attaque en rendant le code plus difficile à analyser. En utilisant la compilation AOT, le code est transformé en un format binaire qui est plus difficile à décompiler et à comprendre, ce qui peut aider à protéger la propriété intellectuelle et à réduire les risques de rétro-ingénierie.

Utilisez la configuration AOT pour compiler et publier le projet Obfuscation. Ensuite, utilisez dotPeek pour analyser le résultat de la compilation AOT et comparez-le avec les résultats précédents. Notez les différences dans la lisibilité du code et la facilité de rétro-ingénierie.

8.6 Activité 6 : Utilisation des options de compilation

Nous allons maintenant étudier quelques options de compilation qui peuvent être utiles pour prévenir des erreurs de programmation fréquentes. Ouvrez la solution GEI771-Labo-Q6.slnx qui est contenue dans un projet C++ dans votre environnement Visual Studio 2026.

Assurez-vous que les options suivantes sont bien configurées:

1. Désactivez les *Basic Runtime Checks* en sélectionnant la valeur "Default"
2. Désactivez aussi l'option *Buffer Security Check* en choisissant la valeur "No (/GS-)"
3. Désactivez les vérifications SDL (/sdl-)
4. Compilez le code pour une architecture x86

8.6.1 Fonctions non-sécuritaires

Lancez la compilation et constatez qu'il en résulte des avertissements. Il faut conserver le code intact pour la poursuite de l'activité. Il faut donc ajouter une définition de pré-préprocesseur pour retirer ces erreurs (voir ici). Vous pouvez donc constater que, par défaut, le compilateur de Visual Studio empêche l'utilisation de fonctions jugées non-sécuritaires.

Maintenant que le code compile, il y a trois scénarios à vérifier:

- scénario de base (*basic*)

- scénario de débordement de tampon (*overflow*)
- scénario d'écrasement de la valeur de l'adresse de retour (*return address smash*)

Les sous-activités suivantes démontrent comment les options de compilation aident à mitiger du code non-sécuritaire.

Note: vous pouvez utiliser les configurations de la solution pour facilement permuter les options

8.6.2 Scénario de base

Exécutez le code en utilisant les valeurs attendues pour le nom d'utilisateur et le mot de passe et confirmez que la vérification réussit.

8.6.3 Basic Runtime Checks

Les *Runtime Checks* (RTC) permettent de détecter des débordements pour une compilation sans optimisation (en Debug). Pour s'y familiariser, utilisez le scénario *overflow* (avec un utilisateur dont le nom est de 16 caractères).

1. Mettez un point d'arrêt avant la copie de la chaîne de caractères
2. Faites un saut d'instruction
3. Remarquez que la condition devient inutile

Pour aider à votre compréhension, vous pouvez vous référer à la figure 2 pour la convention de mise sur la pile et utiliser la fenêtre de mémoire de Visual Studio, ainsi que le *watch* des adresses des variables locales.

8.6.4 Buffer Security Check

Activons maintenant le *security cookie*, qui sert de prévention à l'écrasement de la valeur de l'adresse de retour. Il s'agit de l'option de génération de code *Buffer Security Check*.

1. En utilisant la fenêtre de *Disassembly*, repérez où le cookie est ajouté
2. Repérez quand il est vérifié

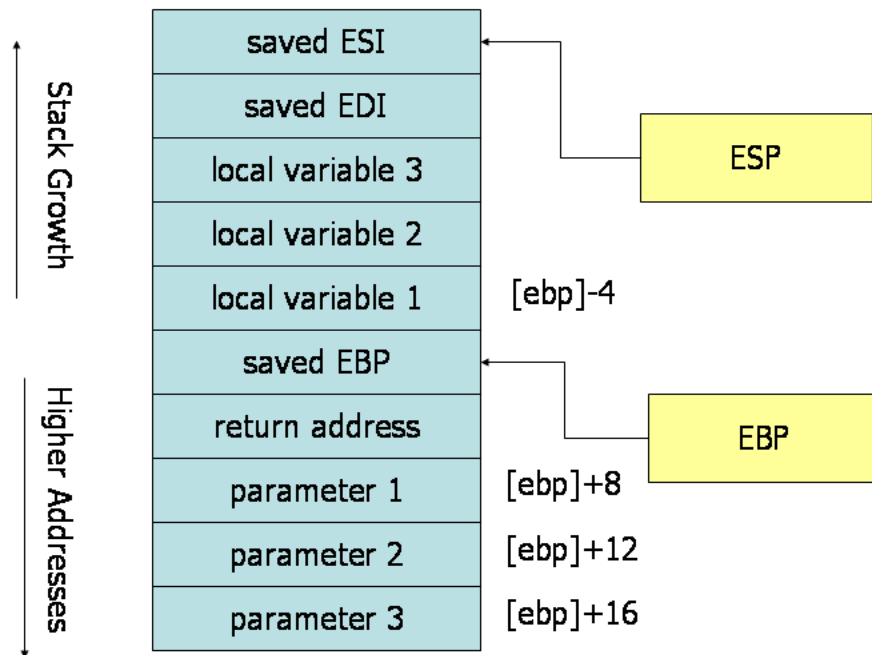


Figure 2: Convention de mise sur la pile

8.6.5 Options SDL

Cette option permet d'empêcher la compilation de code avec de potentielles vulnérabilités de sécurité.

1. Activez l'option sdl
2. Enlever la définition de préprocesseur ajoutée à la première activité
3. Que remarquez-vous?

8.7 Activité 7: Calcul de la couverture de code par les tests

Afin de pouvoir calculer la couverture de code par une batterie de tests, nous utiliserons coverlet. Créer un projet de test avec le gabarit xUnit et vous verrez que le package est présent. Ensuite, utilisez l'option `-collect` pour générer les données d'exécution de la suite de tests et l'outil `dotnet reportgenerator` pour générer un rapport dans le format souhaité.

8.8 Activité 8: Création de la nomenclature logicielle

La nomenclature logicielle (ou *software bill of materials*, SBOM) est un document qui décrit les composants d'un système logiciel, leurs relations et leurs

interactions. Dans le cadre de cette activité, vous allez créer une nomenclature pour le projet sur lequel vous travaillez, afin de décrire les différents modules externes qui le composent. Deux formats existent:

- **SPDX:** Standard axé sur la gestion des licences et la conformité en source libre (supporté par Linux Foundation).
- **CycloneDX:** Standard axé sur la sécurité de la chaîne d'approvisionnement logicielle (support par OWASP).

Pour débiter, nous allons installer les outils dotnet nécessaires à la création de la nomenclature:

- pour générer le format SPDX: sbom-tool
- pour générer le format CycloneDX: dotnet-CycloneDX

Afin de générer des rapports intéressants, nous allons utiliser la solution Obfuscation, à laquelle nous allons ajouter la dépendance à *Newtonsoft.Json*, version 12.0.3, qui contient une vulnérabilité.

Générez le rapport SPDX pour la solution Obfuscation avec l'outil. Ensuite, utilisez le site sbom.sh pour visualiser le rapport. En générant le rapport de vulnérabilités, vous verrez que certaines dépendances ont des vulnérabilités connues. Le standard est par contre plus utilisé pour la gestion de la conformité aux licences. Vous verrez que l'outil a de la difficulté à détecter les licences. Un moyen plus fiable est l'utilisation de la fonctionnalité *Insights* de github, dans la section *Dependency graph*, qui permet d'exporter le SBOM en format SPDX et dans lequel les licences des différentes composantes y figurent.

Générer le rapport CycloneDX pour la solution Obfuscation avec l'outil. Ensuite, utilisez le site sbom.sh pour visualiser le rapport. En générant le rapport de vulnérabilités, vous verrez que certaines dépendances ont des vulnérabilités connues. Un autre outil intéressant pour la visualisation du rapport est Dependency Track. Utilisez le déploiement par docker-compose pour accélérer la mise en route de l'application.

8.9 Activité 9: Ajout de mesures d'atténuation à un modèle de menace

Pour s'initier à l'analyse de la menace au moyen de diagrammes de flux de données et vous aider à choisir celui qui vous préférez pour résoudre la problématique, nous allons étudier les outils suivants:

1. *Microsoft Threat Modeling Tool*, l'outil de modélisation de Microsoft (outil microsoft)

2. *Threat Dragon*, l'outil de modélisation de l'OWASP (threat dragon)

Installer l'outil de modélisation de la menace de Microsoft et prenez le temps de vous familiariser avec l'interface.

Pour *Threat Dragon*, il y a au moins trois façons d'utiliser l'outil, soit l'application web, un conteneur docker ou une application. Utilisez l'installation de votre choix, mais utiliser la dernière version.

Créez un diagramme de flux de données modélisant un fureteur se connectant à une application web exposant une API, qui elle se connecte à un système de fichiers, en utilisant les deux outils. Pour l'outil microsoft, utilisez le gabarit (*template*) SDL. Au fur et à mesure de votre modélisation, vous pouvez ajouter et retirer des menaces.

1. Après votre modélisation, quelles sont les menaces identifiées?
2. Ajoutez une frontière de confiance. Y a-t-il des mises à jour au diagramme?

9 Semaine 2 - Formation à la pratique procédurale

But de l'activité

Le but de ce procédural est de se familiariser avec les techniques de programmation sécurisée et les types de tests qui peuvent être inclus dans un plan de test.

Lectures préalables:

1. [How06, chapitre 12]
2. [Mic11]
3. [Tay08]
4. [How10, extraits]
5. [Net]

9.1 Question 1

Quel est le principe du *fuzzing*, selon [Tay08]? Sur quelles portions d'une application ce type de test se concentre-t-il?

9.2 Question 2

Quel est le principe des tests de pénétration et la différence par rapport au fuzzing?

9.3 Question 3

Avec quelles pratiques de réalisation et de vérification pouvons-nous découvrir chacun des types de bogues mentionnés dans la question précédente?

9.4 Question 4

Expliquez pourquoi il est préférable d'éviter dès le départ d'utiliser le compte administrateur pour implémenter une application et pourquoi cela rend l'utilisation de l'application moins sécuritaire.

9.5 Question 5

Expliquez l'architecture *zero trust* selon [Net] et comment elle s'applique à la résolution de la problématique.

9.6 Question 6

Les questions suivantes sont à propos des pièges de programmation selon [How10, extraits]

N°5: dépassement de tampons

1. Décrivez le piège
2. Donnez des pistes pour détecter le piège
3. Comment s'assurer de ne pas tomber dans le piège

N°6: problème de format de *string*

1. Décrivez le piège, selon le code montré dans la référence
2. Donnez des pistes pour détecter le piège
3. Comment s'assurer de ne pas tomber dans le piège

N°7: dépassement d'entiers

1. Décrivez le piège
2. Donnez des pistes pour détecter le piège
3. Comment s'assurer de ne pas tomber dans le piège

N°9: attraper les exceptions

1. Décrivez le piège
2. Donnez des pistes pour détecter le piège
3. Comment s'assurer de ne pas tomber dans le piège

N°10: injection de commandes

1. Décrivez le piège
2. Donnez des pistes pour détecter le piège
3. Comment s'assurer de ne pas tomber dans le piège

N°11: gérer les erreurs incorrectement

1. Décrivez le piège
2. Donnez des pistes pour détecter le piège
3. Comment s'assurer de ne pas tomber dans le piège

N°12: fuite d'information

1. Décrivez le piège
2. Donnez des pistes pour détecter le piège
3. Comment s'assurer de ne pas tomber dans le piège

N°13: race conditions

1. Décrivez le piège
2. Donnez des pistes pour détecter le piège
3. Comment s'assurer de ne pas tomber dans le piège

N°14: faible utilisabilité

1. Décrivez le piège
2. Donnez des pistes pour détecter le piège
3. Comment s'assurer de ne pas tomber dans le piège

N°15: difficulté de mise à jour

1. Décrivez le piège
2. Donnez des pistes pour détecter le piège
3. Comment s'assurer de ne pas tomber dans le piège

N°16: exécution de code avec des privilèges trop élevés

1. Décrivez le piège
2. Donnez des pistes pour détecter le piège
3. Comment s'assurer de ne pas tomber dans le piège

N°17: échec de la protection des données stockées

1. Décrivez le piège
2. Donnez des pistes pour détecter le piège
3. Comment s'assurer de ne pas tomber dans le piège

N°19: utilisation de système à mots de passe faibles

1. Décrivez le piège
2. Donnez des pistes pour détecter le piège
3. Comment s'assurer de ne pas tomber dans le piège

N°20: nombres aléatoires faibles

1. Décrivez le piège
2. Donnez des pistes pour détecter le piège
3. Comment s'assurer de ne pas tomber dans le piège

N°21: utilisation de la mauvaise cryptographie

1. Décrivez le piège
2. Donnez des pistes pour détecter le piège
3. Comment s'assurer de ne pas tomber dans le piège

10 Semaine 2 - Validation pratique de la solution en laboratoire**Activité de laboratoire**

Lors de la validation, les étapes suivantes seront vérifiées:

1. validation des requêtes contenues dans la collection Postman
2. survol de la modélisation de menace (en préparation du dépôt)
3. évaluation de l'efficacité de l'intégration des mesures de mitigation dans le code source par la batterie de tests

11 Semaine 2 - Travail de rédaction des productions exigées

Rédaction du rapport d'APP suivant les directives données à la section 13.
Finalisation du schéma de concept.

12 Semaine 2 - 2e rencontre de tutorat: validation des connaissances acquises

Validation des connaissances - Bilan de groupe - Travail personnel de synthèse et d'études correctives.

13 Rapport d'APP

13.1 Consignes pour la préparation du dépôt du rapport et de la solution à la problématique

Documents à remettre avant le début de la première rencontre du deuxième tutorat, sur Microsoft Teams.

Voici les documents à rendre, le tout dans un seul fichier zip, selon l'arborescence suivante:

- *répertoire modelisation: un document JSON (pour threat dragon) ou TM7 (pour l'outil microsoft), nommé par les CIP des membres de l'équipe et séparés par des traits d'union (e.g. : abcd1234-efgh5678.json) et comprenant la modélisation de la menace et la description des mesures de mitigation mises en place pour chacun des vecteurs d'attaque et toutes justifications nécessaires à la compréhension de la gestion du risque.*
- *répertoire src: le code source (i.e. excluant les artefacts de compilation) contenu dans une solution Visual Studio 2022, laquelle contient:*
 - *le projet Visual Studio 2022 d'API web ASP.NET incluant le mécanisme d'obfuscation du code*
 - *la batterie de tests automatisés dans un projet Visual Studio 2022 xUnit dotnet*
- *répertoire postman: la collection Postman exportée contenant les requêtes pour les cas d'utilisation de l'application*
- *répertoire sbom: le rapport CycloneDX de la solution*

13.2 Modalités d'évaluation du rapport d'APP

L'évaluation du rapport d'APP contribue à l'évaluation des éléments de compétence de l'unité (voir section suivante). On évalue l'exactitude, la précision, la complétude, la valeur de chaque élément de solution.

La qualité de la communication ne sera pas évaluée de façon sommative mais si le rapport est fautif sur le plan de la qualité de l'écrit et de la présentation, il vous sera retourné pour correction avant qu'il ne soit noté.

14 Évaluation sommative

L'évaluation sommative porte sur tous les objectifs d'apprentissage de l'unité.

15 Évaluation de l'unité

La note attribuée aux activités pédagogiques de l'unité est une note individuelle. L'évaluation portera sur les compétences figurant dans la description des activités pédagogiques. Ces compétences, ainsi que la pondération de chacune d'entre elles dans l'évaluation de cette unité, sont :

GEI-771 - Programmation sécurisée				
Activités et éléments de compétence	Validation	Rapport	Examens	
			sommatif	final
Compétence 1	25	35	105	135
Compétence 2	25	35	105	135
Total:	600			

Seuil des cotes											
E	D	D+	C-	C	C+	B-	B	B+	A-	A	A+
<	50%	54%	58%	62%	66%	70%	74%	78%	82%	86%	90%
0	1.0	1.3	1.7	2.0	2.3	2.7	3.0	3.3	3.7	4	4.3

15.1 Utilisation de l'intelligence artificielle générative

Dans le cadre de la présente activité, vous êtes réputés avoir pris connaissance de la page *Intégrité intellectuelle des Services à la vie étudiante*. L'intelligence artificielle générative (IAG) réfère à l'ensemble d'outils capable de générer de nouveaux contenus (texte, images, code informatique, musique, etc.) à partir de requêtes texte, incluant, mais pas limité à :

- ChatGPT (fin de l'apprentissage en 2021)
- CoPilot

- Dall-E
- Synthesia
- Scribe
- Duet
- Bard

L'écosystème des IAG est extrêmement énergivore et a un impact environnemental significatif. Avant d'avoir recours à ces outils, vérifiez si la réponse à votre requête pourrait se trouver dans votre documentation, vos collègues, votre équipe professorale ou par l'utilisation d'un moteur de recherche (Ecosia, Google, etc.) afin de limiter votre empreinte énergétique.

Attention: il peut y avoir des erreurs de l'IAG.

La responsabilité finale du contenu des productions appartient à la personne étudiante.

Dans le cadre de cette unité d'APP, l'usage des IAG est permis pour votre apprentissage, l'aide à la rédaction et pour résoudre la problématique, conditionnellement à ce que tout élément produit par une IAG directement utilisé dans les productions remises à l'équipe professorale soit cité et documenté dans les règles de l'art.

L'utilisation des IAG lors des évaluations sommatives et finales n'est pas permise.